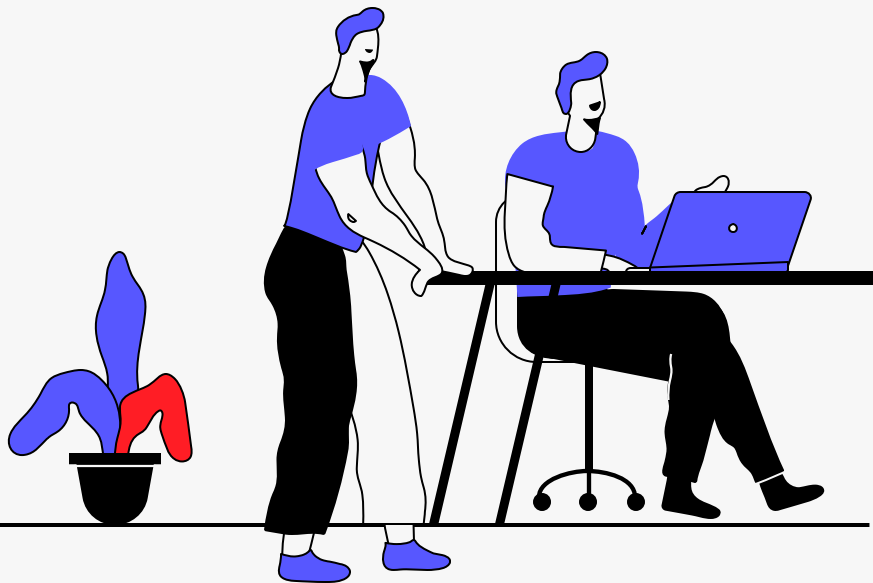


Debuggeando en Visual Studio Code



Introducción a la Programación
Orientada a Objetos





Utilizando el debugger

- Todos los ambientes de desarrollo incorporan herramientas de depuración de código. **Visual Studio Code** no es la excepción.
- Un debugger o depurador de código es una herramienta que nos va a permitir ejecutar nuestro código en un modo especial para:
 - seguir la ejecución del programa de manera detallada y observar el flujo de control de nuestro programa
 - observar el contenido de variables, parámetros y atributos
 - examinar la pila de llamadas del sistema (call stack) para observar el contexto de ejecución
 - observar los objetos en ejecución y sus referencias





Utilizando el debugger

- Un debugger o depurador de código es una herramienta que nos va a permitir ejecutar nuestro código en un modo especial para:
 - controlar la ejecución deteniéndola en puntos específicos (breakpoints)
 - controlar el contenido de memoria pudiendo modificar entidades de tipos elementales
 - evaluar expresiones arbitrarias y condiciones (watch)
 - aplicar técnicas de debugging avanzadas que no veremos en esta introducción
- **Y todo lo anterior SIN ENSUCIAR/MODIFICAR nuestro código!**





Pasos para debuggear

1. Iniciar el modo de depuración (debugger) del IDE.
 - El código debe compilar correctamente
2. Establecer correctamente el contexto de depuración (i.e. llegar al lugar del código donde quiero depurar y en el contexto de ejecución adecuado)
 - Establecer los breakpoints adecuados
 - Verificar que la ejecución se da en un contexto de invocación de métodos relevante para el objetivo de depuración (ver el call stack)
3. Examinar el flujo de ejecución (step over/step into/step out) y el contenido de memoria (variables/watch). Verificar si son los esperados (ok) o si difieren (bugs).
4. En caso de ser necesario, modificar el contenido de memoria para llevarlo a lo esperado y poder continuar depurando.



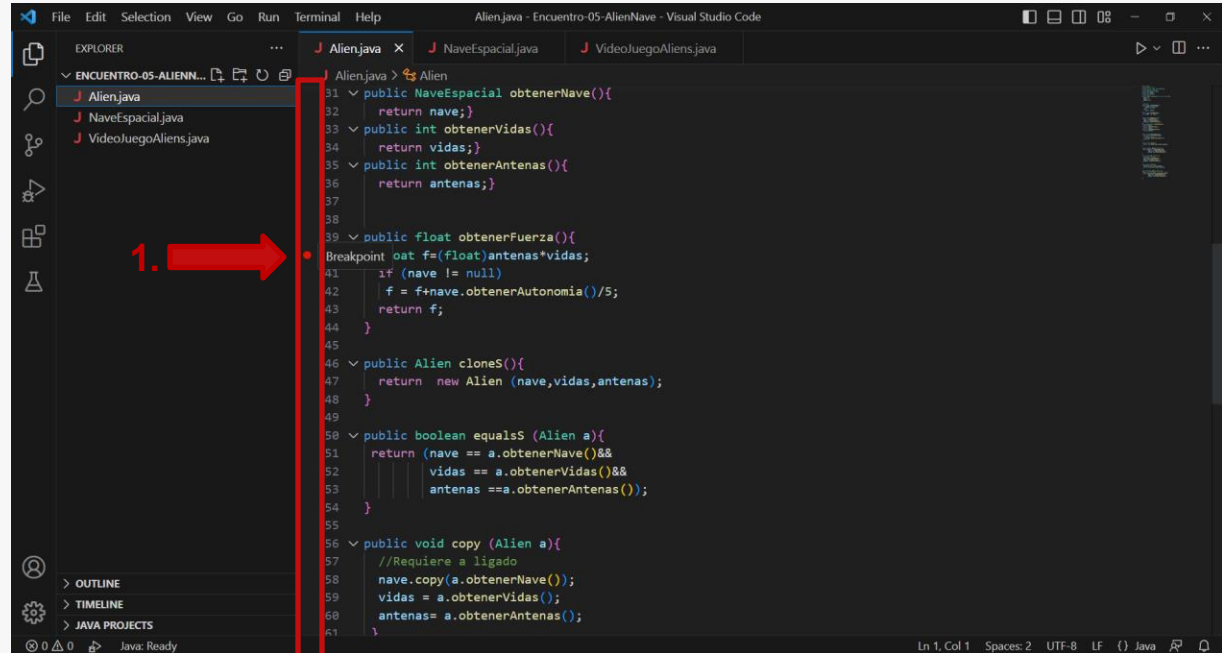
Iniciando el debugger en VS Code

1. Inserta al menos un Breakpoint (punto de ruptura) en alguna línea de código ejecutable que sea alcanzada por la ejecución del código. Hacer click en el margen indicado, en la línea seleccionada, para que aparezca el punto rojo.

Son líneas ejecutables de código aptas para insertar breakpoints:

- Asignaciones
- Instrucciones en las que se evalúen expresiones
- Instrucciones en las que se envíen mensajes a objetos

No se insertan breakpoints en líneas en blanco, comentarios ni declaraciones.

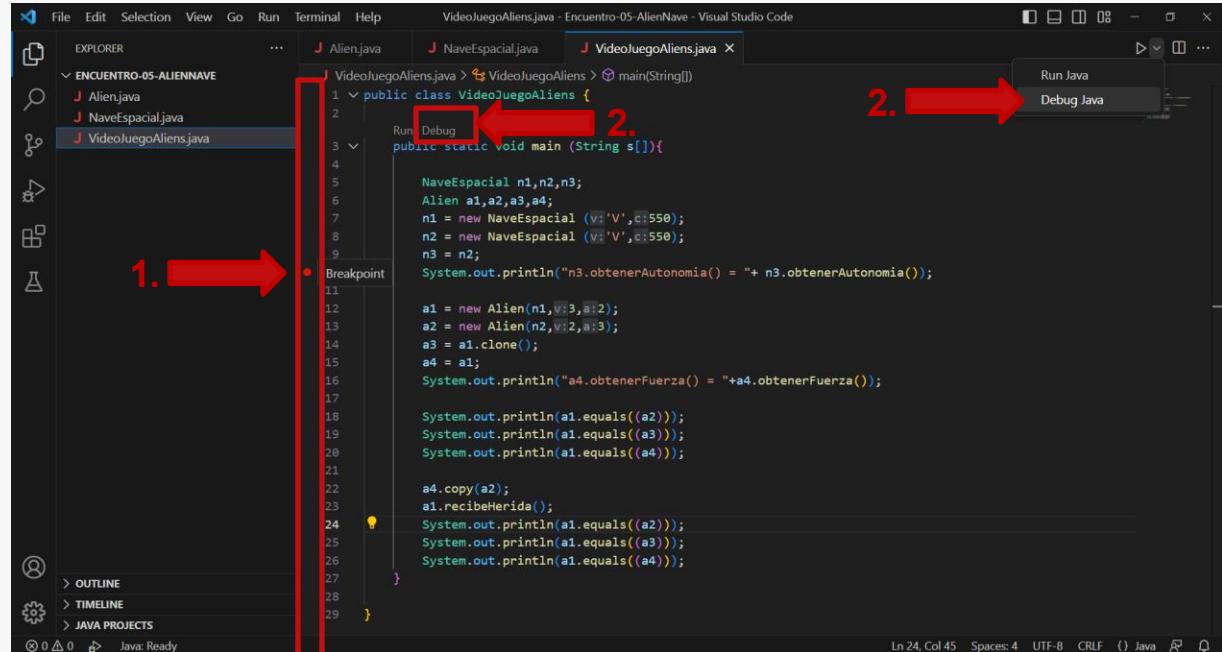


Iniciando el debugger en VS Code

2. Dirígete a la clase con el método main y selecciona **Debug** en la línea superior al encabezado de main, o **Debug Java** del menú de ejecución.

Esto inicia la ejecución del programa en modo debug. La misma se detendrá en el primer breakpoint que se alcance.

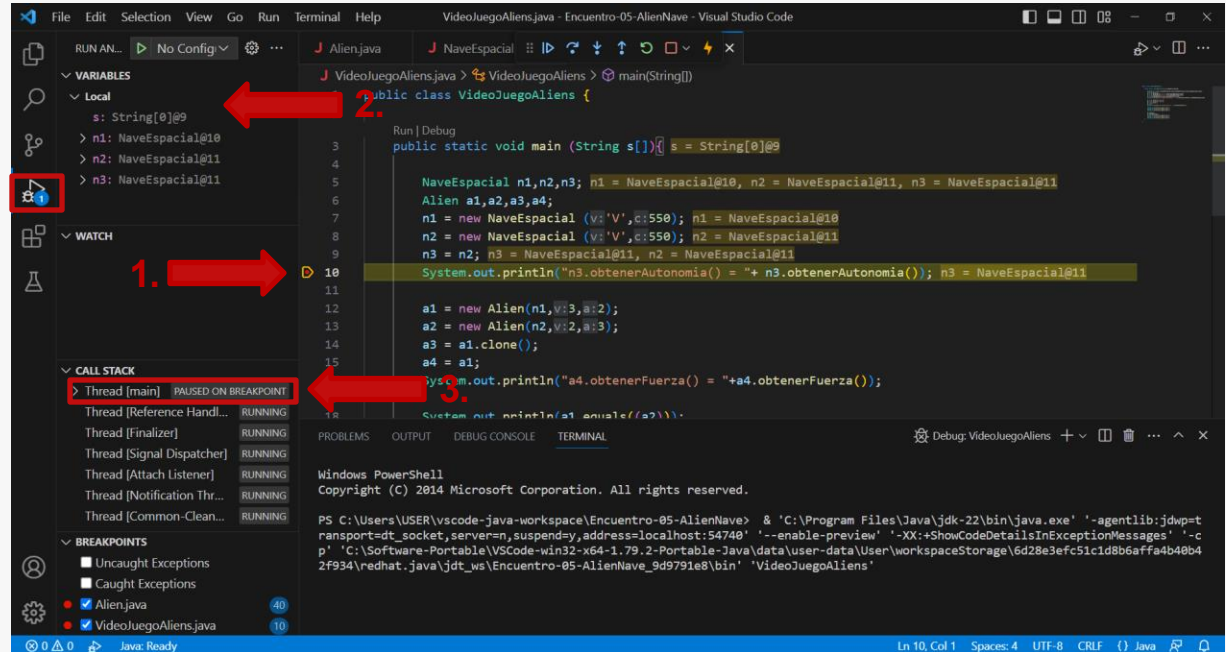
Si no se alcanza ningún breakpoint, la ejecución progresará hasta su finalización (como en el caso de una ejecución normal)



Iniciando el debugger en VS Code

3. La vista de Run & Debug de VS Code muestra información adicional de depuración en la sesión de depuración actual.

1. Se indica/resalta la línea de ejecución actual
2. Variables: incluye el contenido de memoria (variables, parámetros, atributos)
3. Call Stack: las pilas de invocaciones de los servicios de Java (muestra cosas del subsistema de Java ejecutadas a la par de nuestro proyecto)



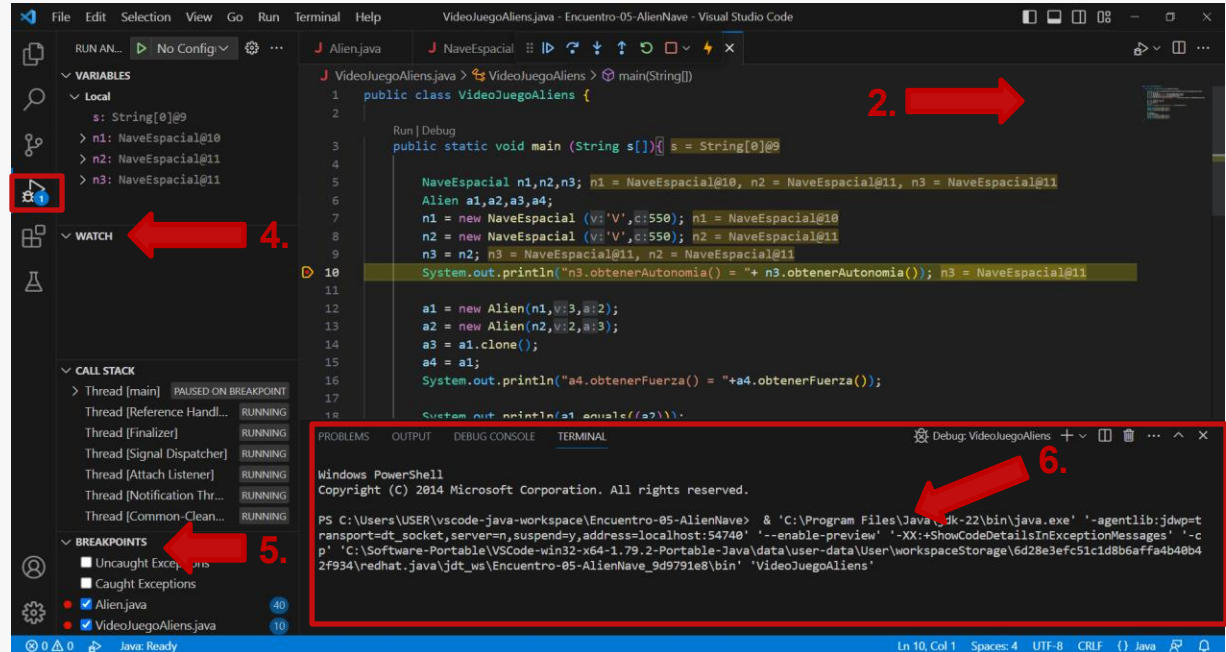
Iniciando el debugger en VS Code

3. La vista de Run & Debug de VS Code muestra información adicional de depuración en la sesión de depuración actual.

4. Watch: es para evaluar expresiones dinámicamente

5. Breakpoints: Contiene un listado de los breakpoints que insertamos en el proyecto, para activarlos, desactivarlos, configurarlos o eliminarlos.

6. La consola (Terminal), al igual que en ejecución normal, permite hacer Entrada/Salida con el usuario.



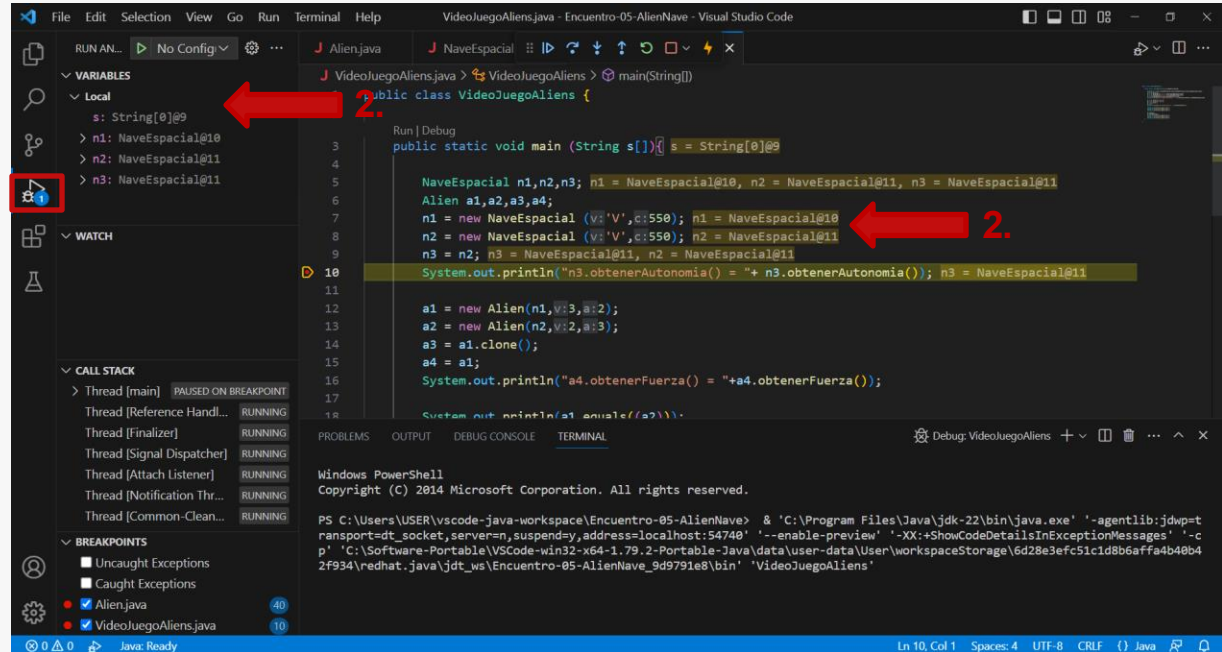
Iniciando el debugger en VS Code

3. La vista de Run & Debug de VS Code muestra información adicional de depuración en la sesión de depuración actual.

2. Variables: incluye el contenido de memoria (variables, parámetros, atributos)

– Se muestran también como anotaciones en el código los valores asignados

– Para objetos la notación en gral. es Clase@ID donde ID es un número relacionado con la dirección de memoria del objeto referenciado (i.e. su identidad).

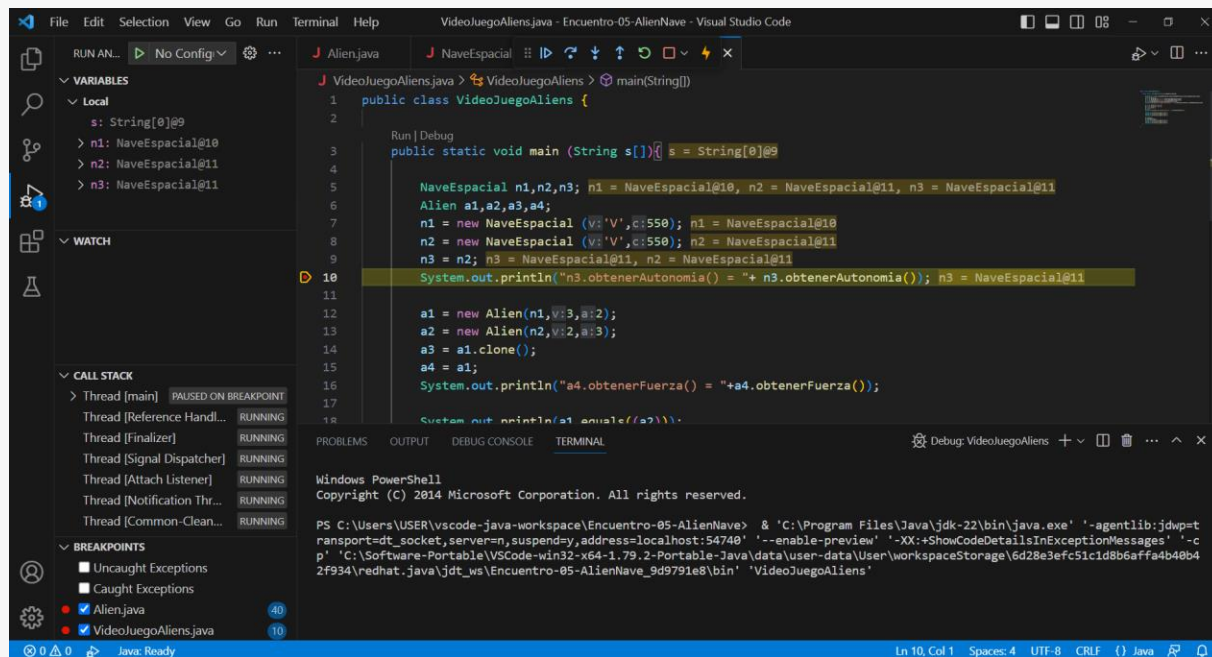


Iniciando el debugger en VS Code

3. La vista de Run & Debug de VS Code muestra información adicional de depuración en la sesión de depuración actual.

En el ejemplo la ejecución se encuentra detenida en la línea 10. Ya se ejecutaron las líneas 7, 8 y 9. Esto se puede ver en **Variables**.

Local es el ambiente de referenciamiento local al método actual (main), e incluye sus parámetros formales (s) y sus variables locales (n1, n2 y n3 son las únicas inicializadas hasta el momento).

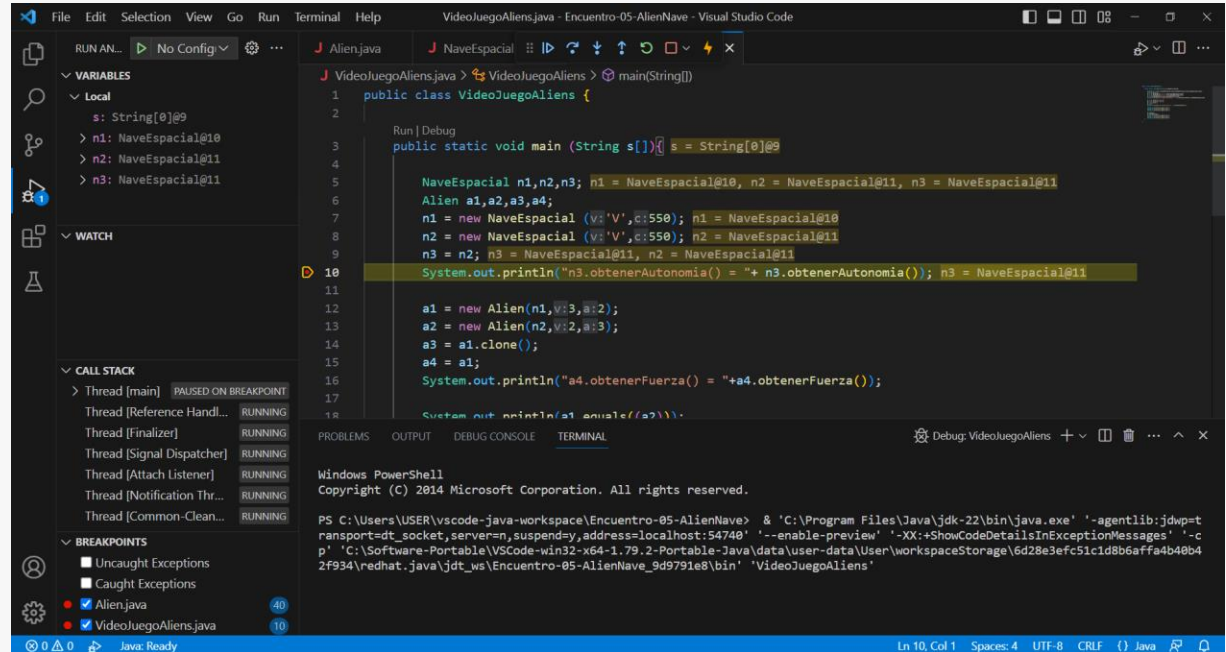


Iniciando el debugger en VS Code

3. La vista de Run & Debug de VS Code muestra información adicional de depuración en la sesión de depuración actual.

n1 mantiene la referencia al objeto NaveEspacial@10, mientras que tanto n2 como n3 mantienen referencias al objeto NaveEspacial@11 (como consecuencia de la asignación de línea 9).

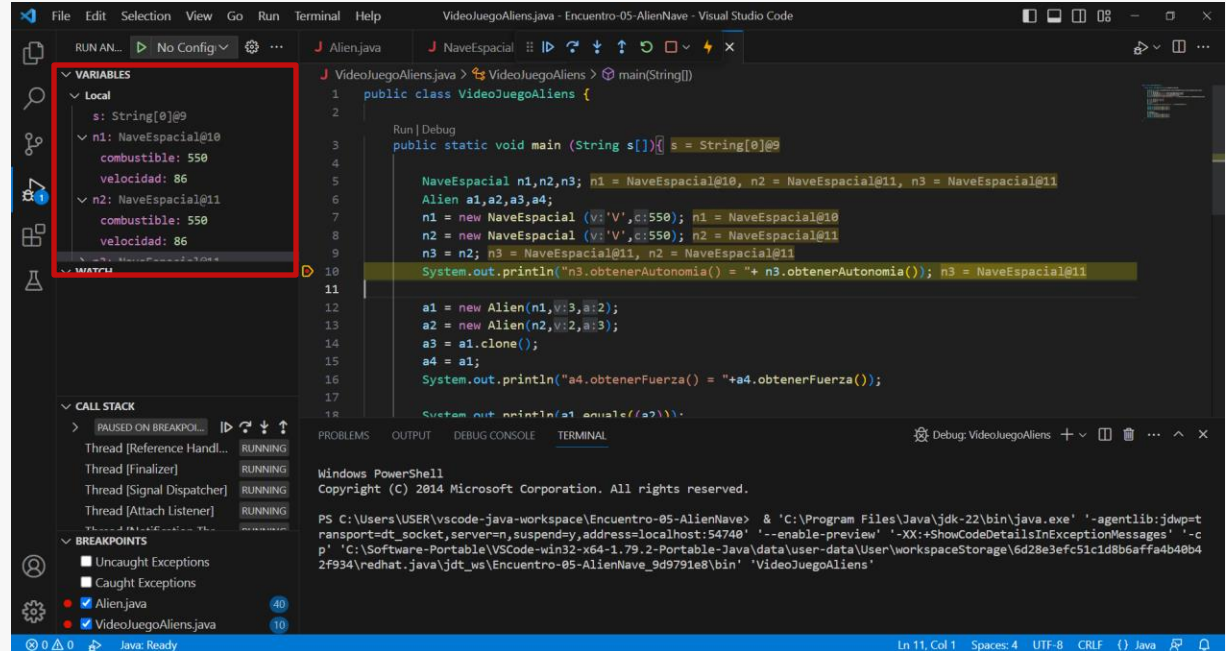
Esto también se puede ver en las anotaciones del código (junto a cada declaración y asignación se actualiza el valor de las variables como en una traza)



Iniciando el debugger en VS Code

3. La vista de Run & Debug de VS Code muestra información adicional de depuración en la sesión de depuración actual.

Los objetos en las Variables, pueden desplegarse para examinar sus atributos.

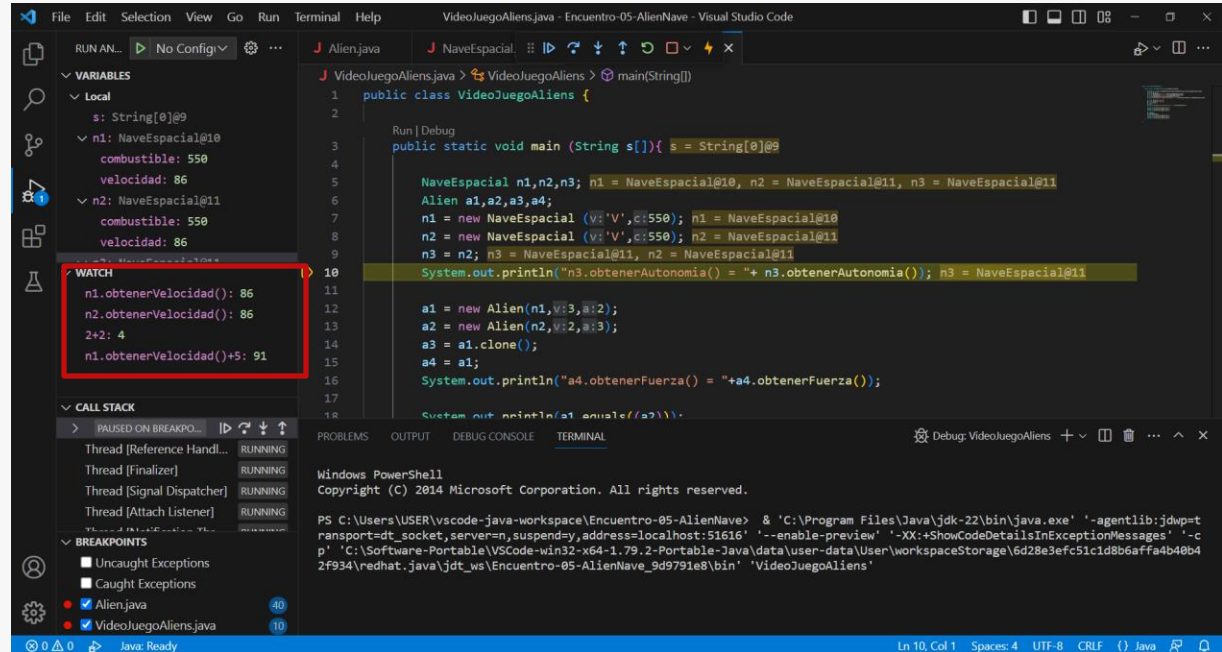


Iniciando el debugger en VS Code

3. La vista de Run & Debug de VS Code muestra información adicional de depuración en la sesión de depuración actual.

Si queremos obtener detalles específicos de los objetos, podemos especificar expresiones de Watch (por ej. si queremos observar las velocidades de los objetos ligados a n1 y n2 sin desplegar todos los objetos en Variables).

Se admite cualquier expresión válida en Java (ej. $2+2$, invocación de consultas, etc.).



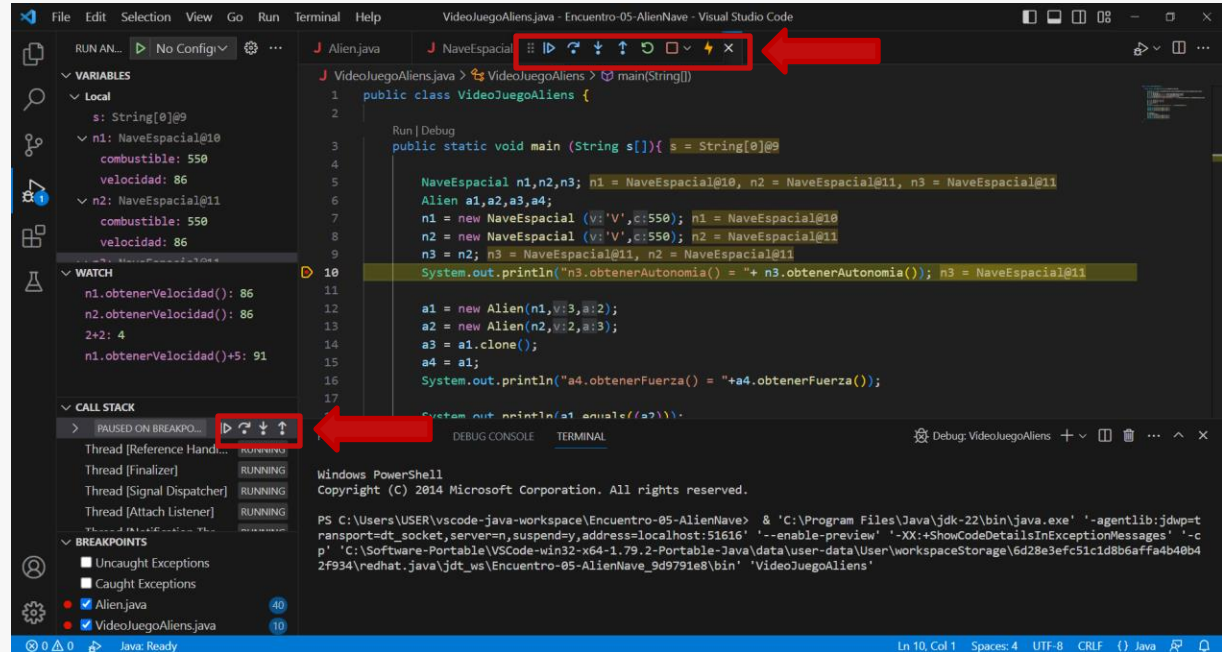
Controlando la ejecución

4. El control de la ejecución se realiza con la barra de herramientas de debug de la parte superior. Algunas opciones también están disponibles en el panel de Call Stack.

Continue (F5): Continúa la ejecución hasta alcanzar un breakpoint o terminar.

Step Over (F10): Avanza la ejecución a la siguiente línea de código ejecutable sin ingresar en los métodos invocados.

Step Into (F11): Ingresa al siguiente método invocado (según el orden de evaluación) y detiene la ejecución en la primera línea ejecutable del mismo.



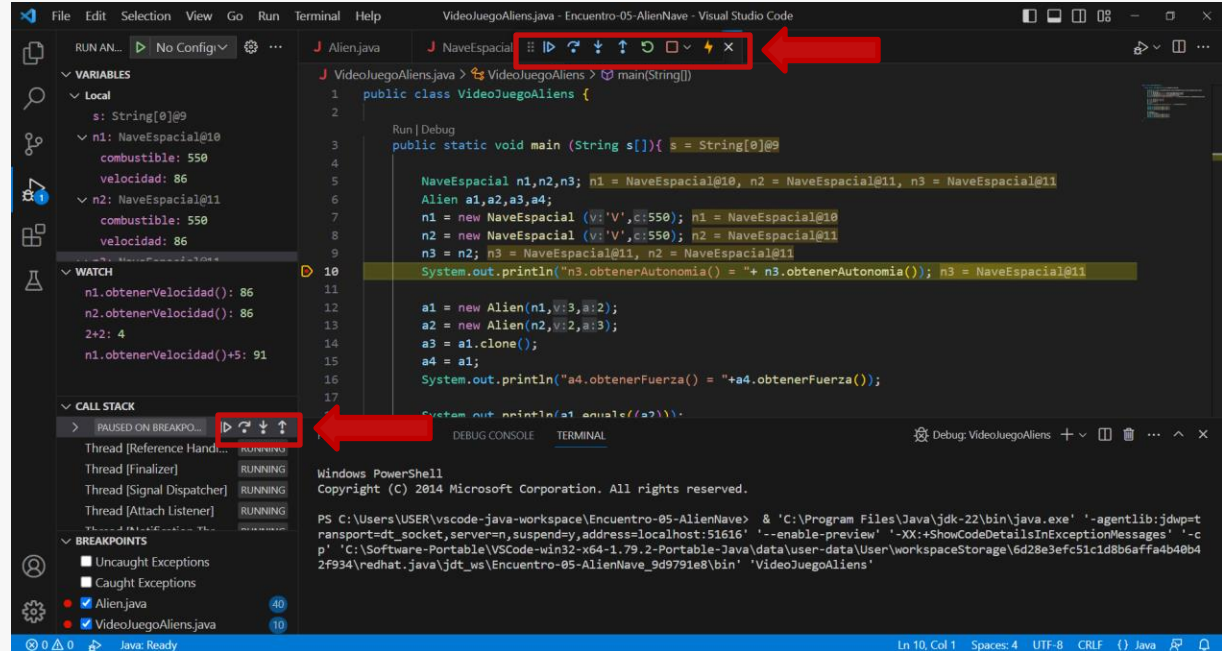
Controlando la ejecución

4. El control de la ejecución se realiza con la barra de herramientas de debug de la parte superior. Algunas opciones también están disponibles en el panel de Call Stack.

Step Out (F12): Continúa la ejecución del método actual hasta su retorno y detiene la ejecución en el método que lo invocó justo luego de la invocación.

Restart (Ctrl+Shift+F5): Reinicia la sesión de debug desde el comienzo.

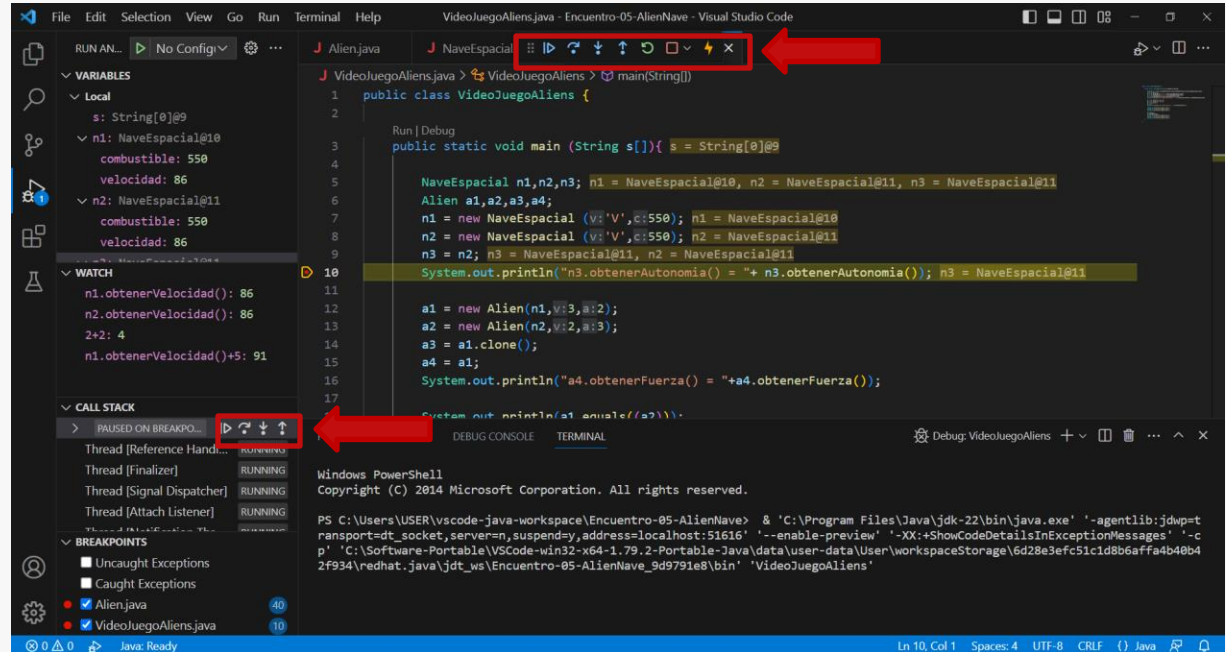
Stop (Shift+F5): Finaliza la sesión de debug actual.



Controlando la ejecución

4. El control de la ejecución se realiza con la barra de herramientas de debug de la parte superior. Algunas opciones también están disponibles en el panel de Call Stack.

La depuración es un proceso hacia adelante. No tengo forma de retroceder la ejecución a un estado anterior. Si "me pasé" del punto de ejecución a analizar, debo reiniciar la sesión con **Restart** (Ctrl+Shift+F5), o detenerla con **Stop** (Shift+F5) para recomenzarla luego.

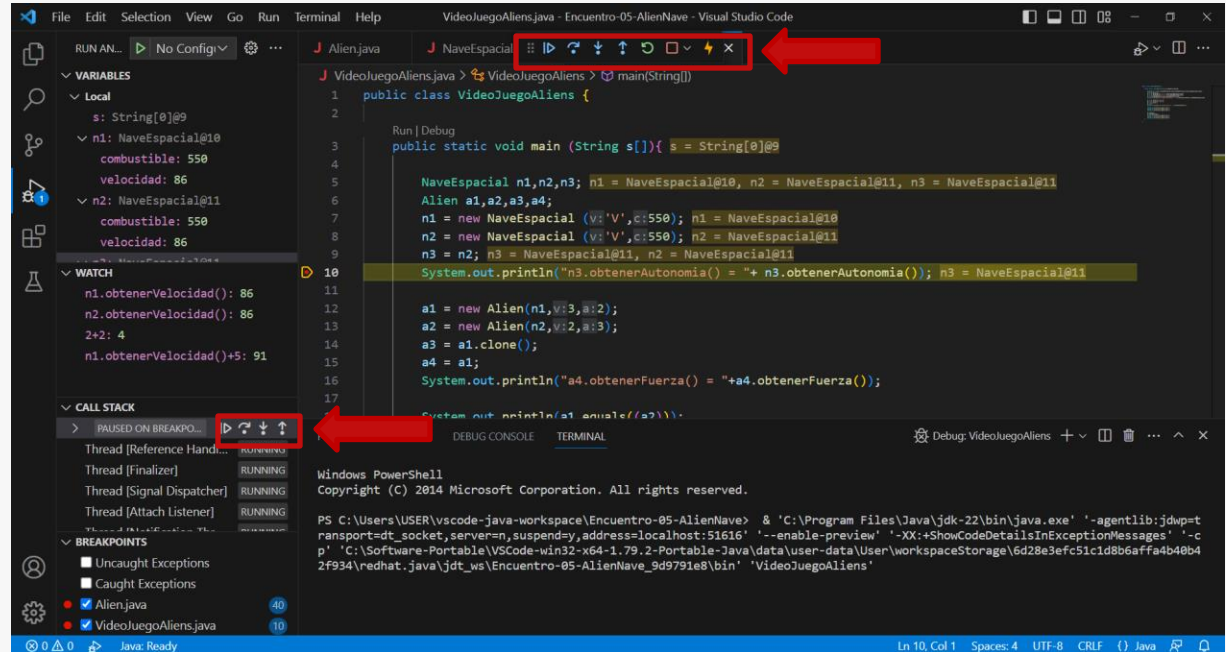


Controlando la ejecución

4. El control de la ejecución se realiza con la barra de herramientas de debug de la parte superior. Algunas opciones también están disponibles en el panel de Call Stack.

La ejecución paso a paso (Step Over (F10), Step Into (F11), Step Out (F12)) nos permite examinar qué camino se toma en los condicionales, evaluar cómo iteran los loops, y seguir de cerca el flujo de ejecución.

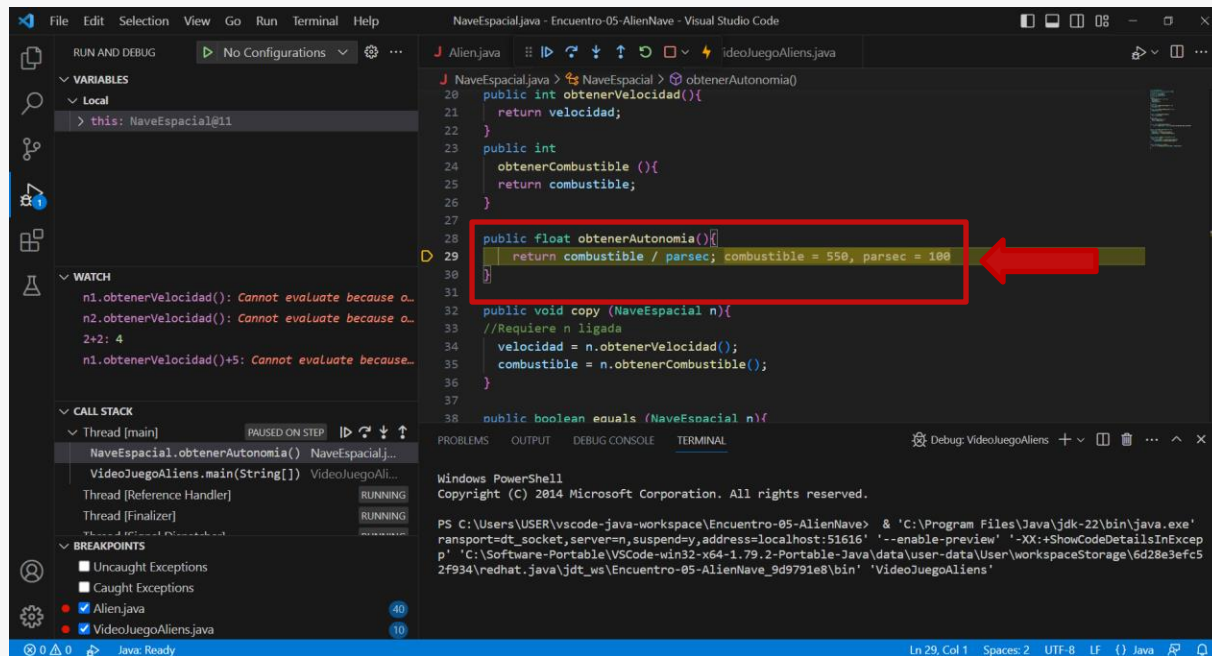
Siempre que se alcance un Breakpoint, la ejecución se interrumpe (aún durante Step Over o Step Out).



Controlando la ejecución

4. El control de la ejecución se realiza con la barra de herramientas de debug de la parte superior. Algunas opciones también están disponibles en el panel de Call Stack.

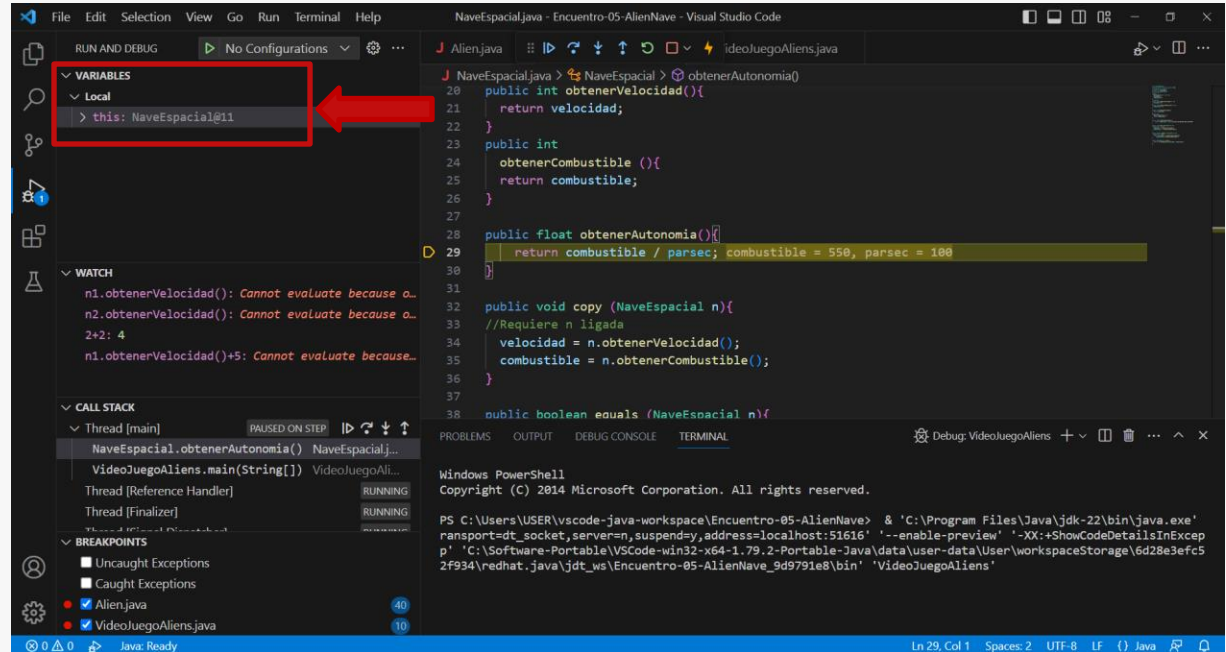
Al hacer Step into, el contexto cambia: pasamos a observar la ejecución del método `obtenerAutonomia()` en respuesta al mensaje enviado al objeto ligado a `n3`. La ejecución se encuentra detenida en la línea 29 de `NaveEspacial.java`, a punto de evaluar la división. El debugger nos anota los valores relevantes para ayudarnos en esa evaluación (`550/100`).



Controlando la ejecución

4. El control de la ejecución se realiza con la barra de herramientas de debug de la parte superior. Algunas opciones también están disponibles en el panel de Call Stack.

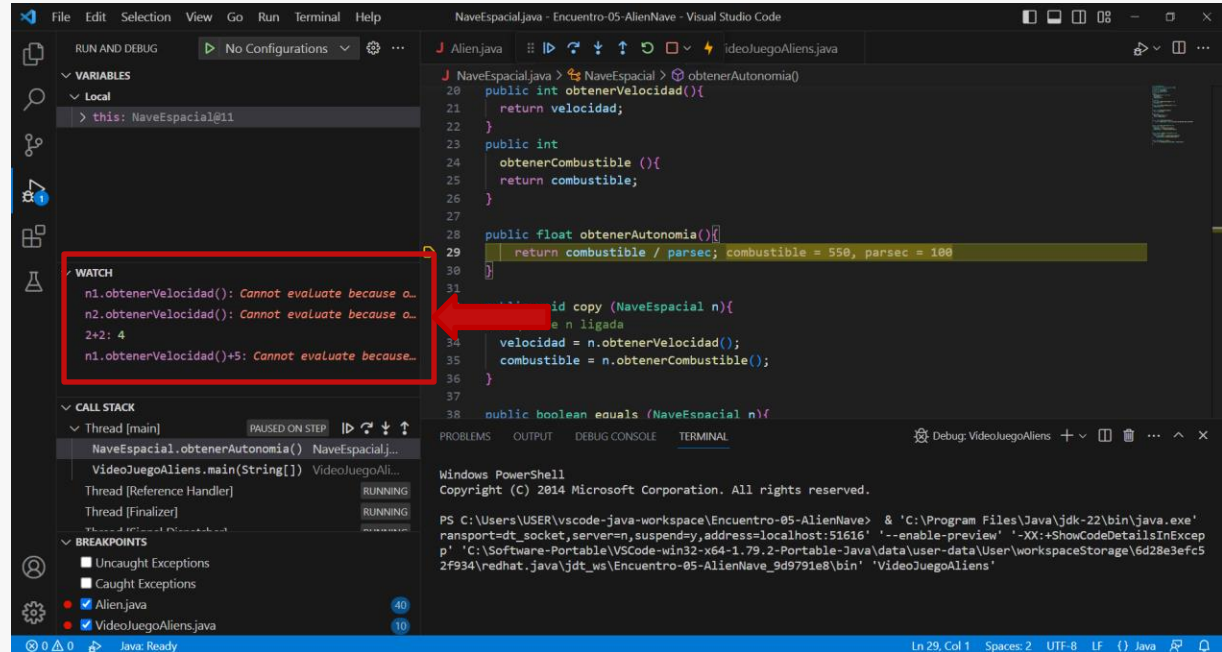
Las variables y el ambiente **Local**, ahora son relativos al método `obtenerAutonomia()`. Dado que no hay parámetros formales ni variables locales, sólo disponemos de la referencia `this` al objeto actual (n3 mantenía la referencia a `NaveEspacial@11` al momento de la invocación).



Controlando la ejecución

4. El control de la ejecución se realiza con la barra de herramientas de debug de la parte superior. Algunas opciones también están disponibles en el panel de Call Stack.

Con excepción de $2+2 = 4$, las expresiones de Watch no pueden ser evaluadas por estar fuera de contexto (el ambiente de referencia-miento del método obtenerAutonomia() no contiene n1 ni n2, ya que esas son variables de main). Es usual que ocurra esto al ingresar a métodos invocados. Al retornar se vuelven a evaluar normalmente.

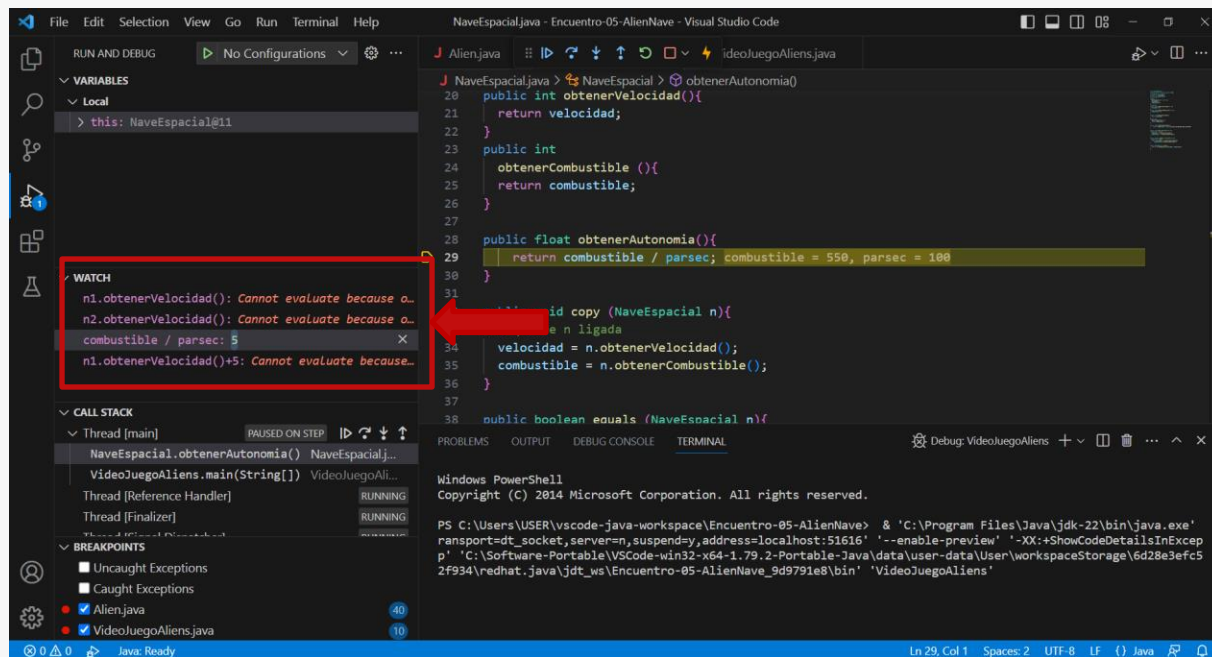


Controlando la ejecución

4. El control de la ejecución se realiza con la barra de herramientas de debug de la parte superior. Algunas opciones también están disponibles en el panel de Call Stack.

Algo útil a realizar para corroborar que todo marcha bien, puede ser añadir una expresión de Watch para evaluar la división combustible/parsec antes de que efectivamente se produzca en la ejecución. Luego podremos corroborar ese valor con lo devuelto por el método.

¿Por qué no resulta 5.5?



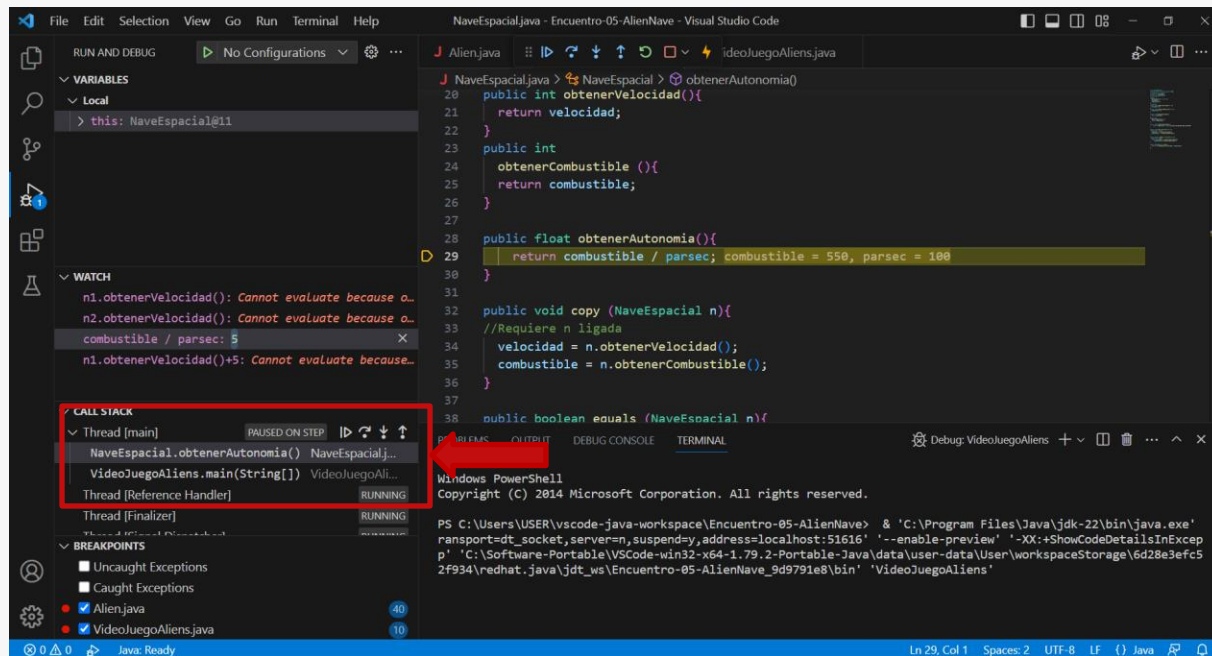
Controlando la ejecución

4. El control de la ejecución se realiza con la barra de herramientas de debug de la parte superior. Algunas opciones también están disponibles en el panel de Call Stack.

Lo interesante ocurre en el Call Stack.

Nuestro código ejecuta en el Thread[main]. Los demás son cosas que ejecuta Java a la vez que nuestro código y que escapan al alcance de la materia.

Desplegando dicho Thread, observamos la pila de llamadas de nuestro código.

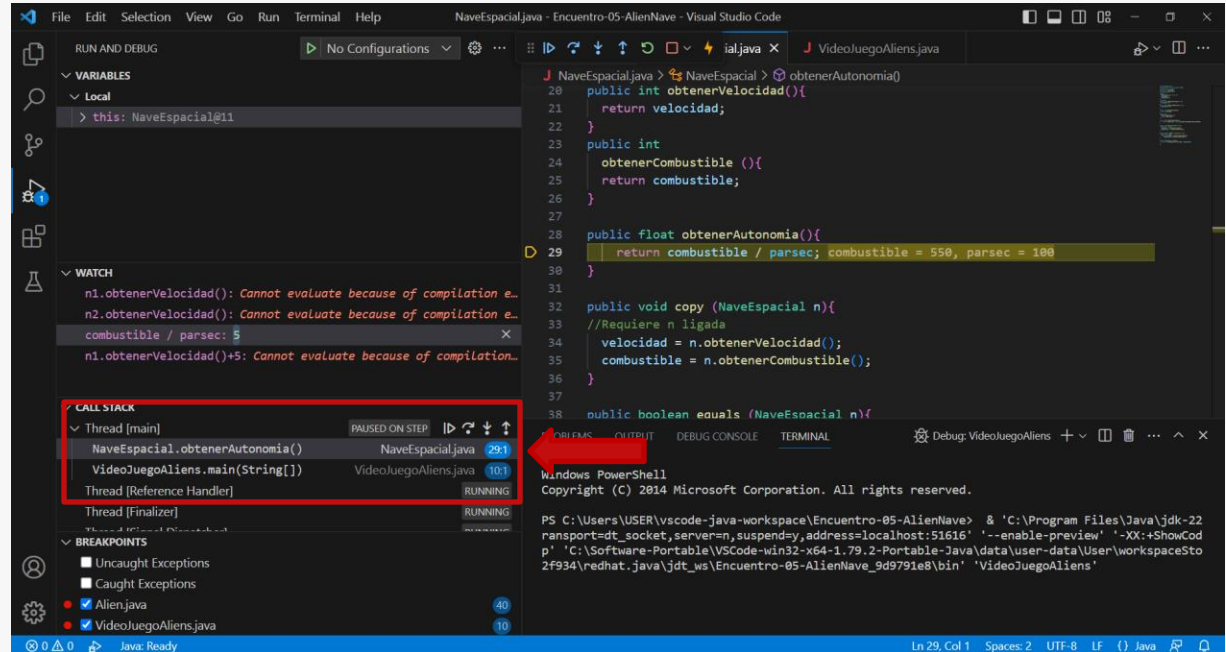


Controlando la ejecución

4. El control de la ejecución se realiza con la barra de herramientas de debug de la parte superior. Algunas opciones también están disponibles en el panel de Call Stack.

En el Call Stack se pueden observar todos los métodos que están en ejecución al momento de la detención:

- main(...) de la clase VideoJuegoAliens detenido en la línea 10 que invocó a
- obtenerAutonomia() de la clase NaveEspacial detenido en la línea 29

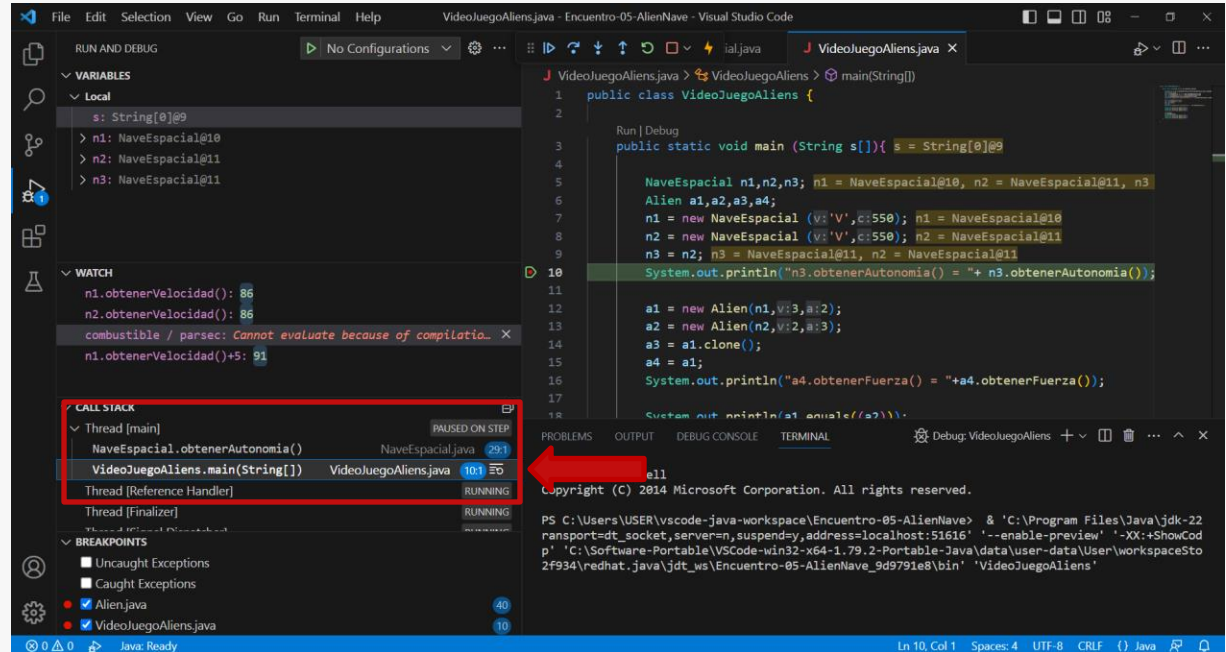


Controlando la ejecución

4. El control de la ejecución se realiza con la barra de herramientas de debug de la parte superior. Algunas opciones también están disponibles en el panel de Call Stack.

El Call Stack nos brinda el contexto de ejecución de un método. Estamos analizando obtenerAutonomia() pero en el contexto en el que fue invocado desde main línea 10.

Haciendo click en cada línea del Call Stack, podemos alternar el contexto y examinar en qué estado se encuentran los métodos invocadores/clientes.

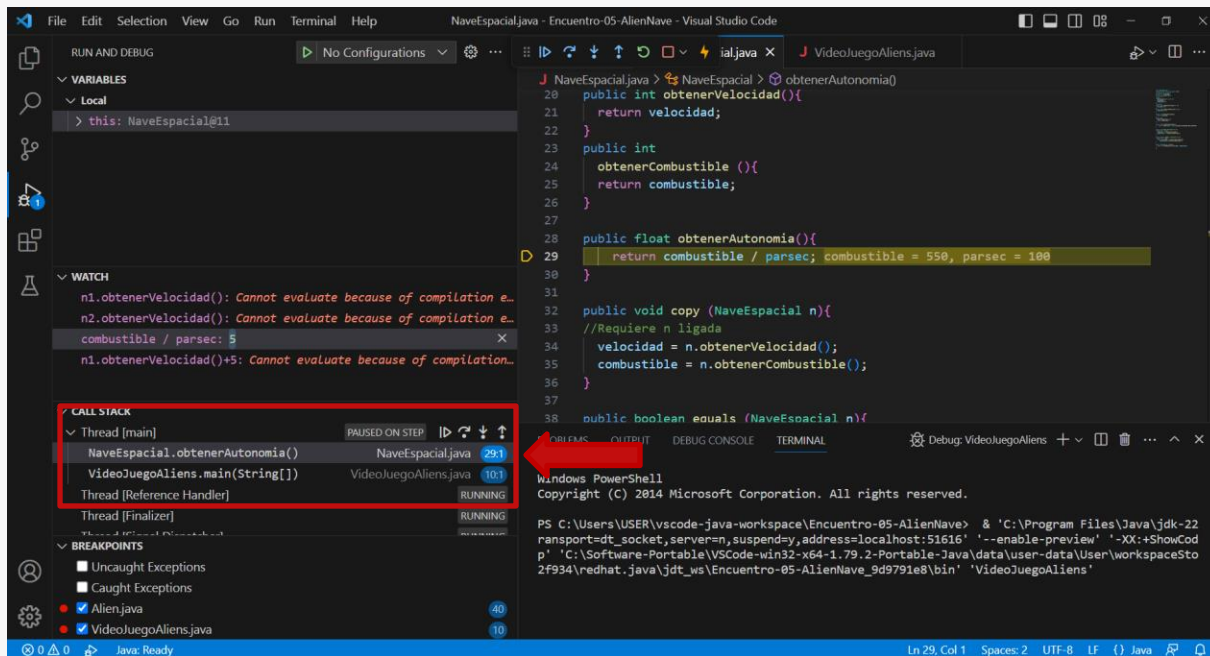


Controlando la ejecución

4. El control de la ejecución se realiza con la barra de herramientas de debug de la parte superior. Algunas opciones también están disponibles en el panel de Call Stack.

El Call Stack es fundamental para establecer el contexto de ejecución de un método (quiénes lo invocaron y en qué estado).

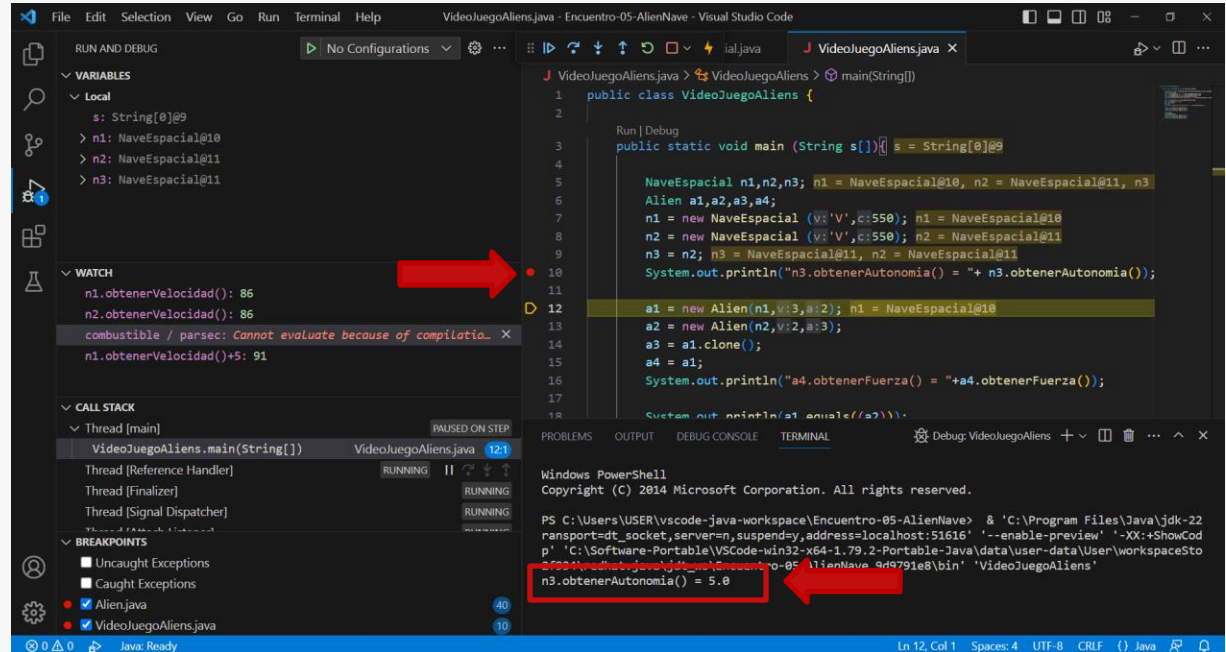
Se utiliza mucho también en el reporte de errores de Java para dar contexto del error/excepción, mediante la traza de pila (de llamadas) o Stack Trace.



Controlando la ejecución

4. El control de la ejecución se realiza con la barra de herramientas de debug de la parte superior. Algunas opciones también están disponibles en el panel de Call Stack.

Al continuar la ejecución un par de pasos, la misma retorna a main, completa la ejecución de la línea 10 mostrando por consola la autonomía calculada. Si no somos cuidadosos acerca de a qué métodos entramos, podríamos terminar examinando métodos de Java como por ej. println, que no necesitamos ver.



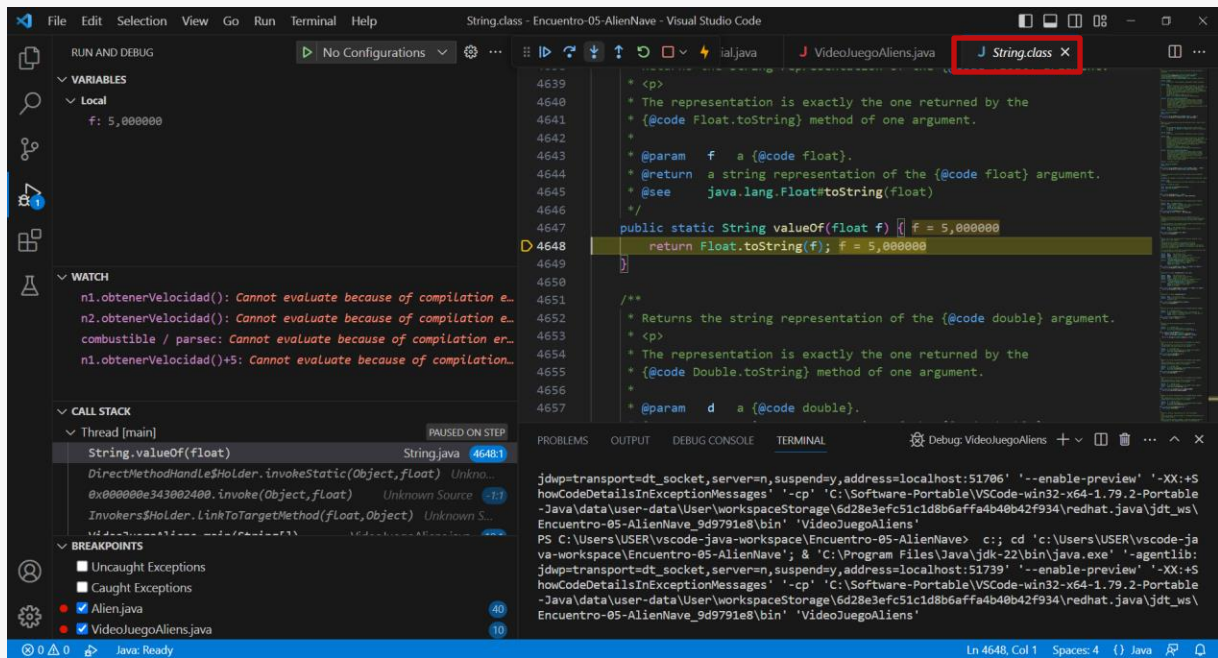
Controlando la ejecución

4. El control de la ejecución se realiza con la barra de herramientas de debug de la parte superior. Algunas opciones también están disponibles en el panel de Call Stack.

Cosas que no necesitamos ver:

El código de conversión de float a String (como paso previo a concatenar Strings en línea 10 de main)

Cuando no se dispone de código fuente, el Debugger nos reconstruye el código a partir de un archivo .class que contiene el resultado de compilarlo (JVM bytecodes).

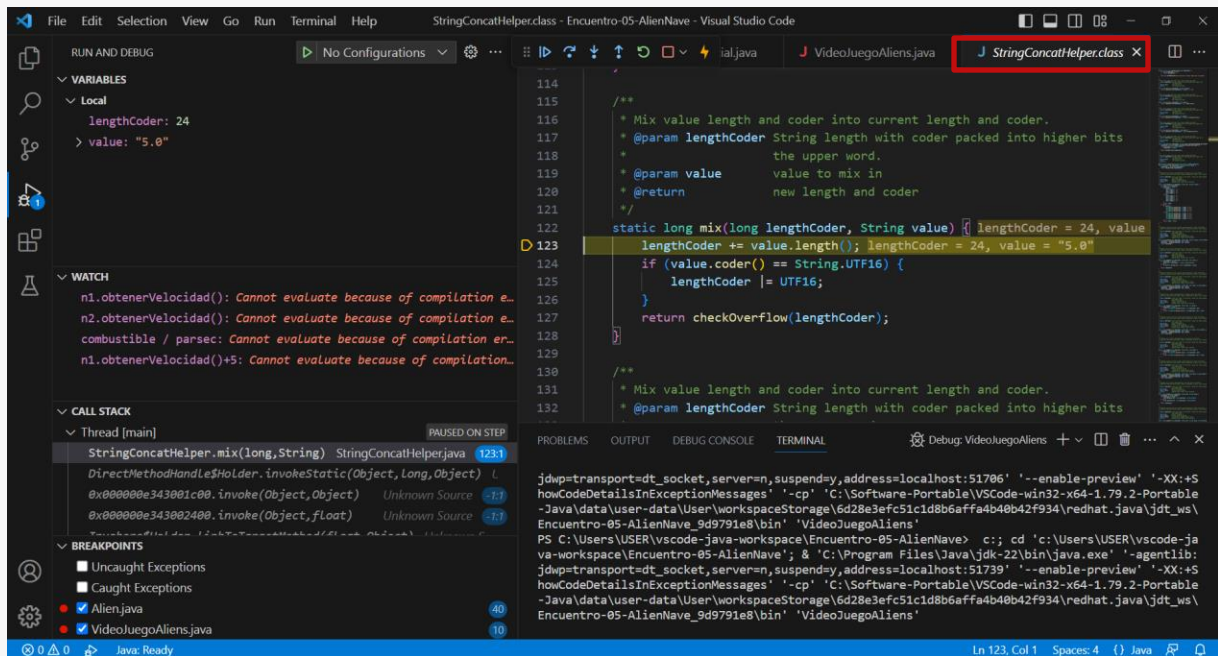


Controlando la ejecución

4. El control de la ejecución se realiza con la barra de herramientas de debug de la parte superior. Algunas opciones también están disponibles en el panel de Call Stack.

Cosas que no necesitamos ver:

El código de concatenación de Strings (como paso previo al println en línea 10 de main)



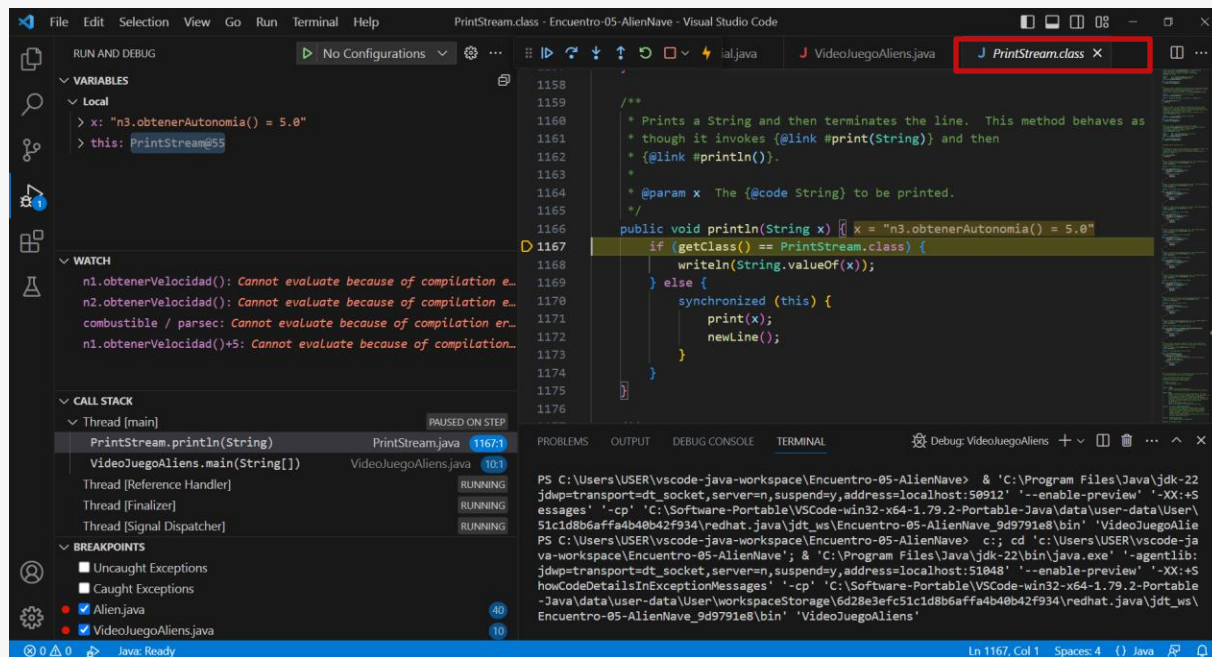
Controlando la ejecución

4. El control de la ejecución se realiza con la barra de herramientas de debug de la parte superior. Algunas opciones también están disponibles en el panel de Call Stack.

Cosas que no necesitamos ver:

El código del println
invocado en línea 10 de
main.

El Debugger es capaz de
mostrarnos todo esto!





Controlando la ejecución

4. El control de la ejecución se realiza con la barra de herramientas de debug de la parte superior. Algunas opciones también están disponibles en el panel de Call Stack.

¿Qué hacer si por error ingresamos en código que no deseamos ver?

- Analizar el contexto (Call Stack) y continuar la ejecución tratando de salir de ahí (Step Out)
- Establecer un Breakpoint temporal en un punto posterior a la invocación del código al que se ingresó, para continuar (Continue) la ejecución hasta el mismo
- Simplemente, reiniciar la sesión de debugging y repetir los pasos para volver al contexto de ejecución previo a la invocación no deseada

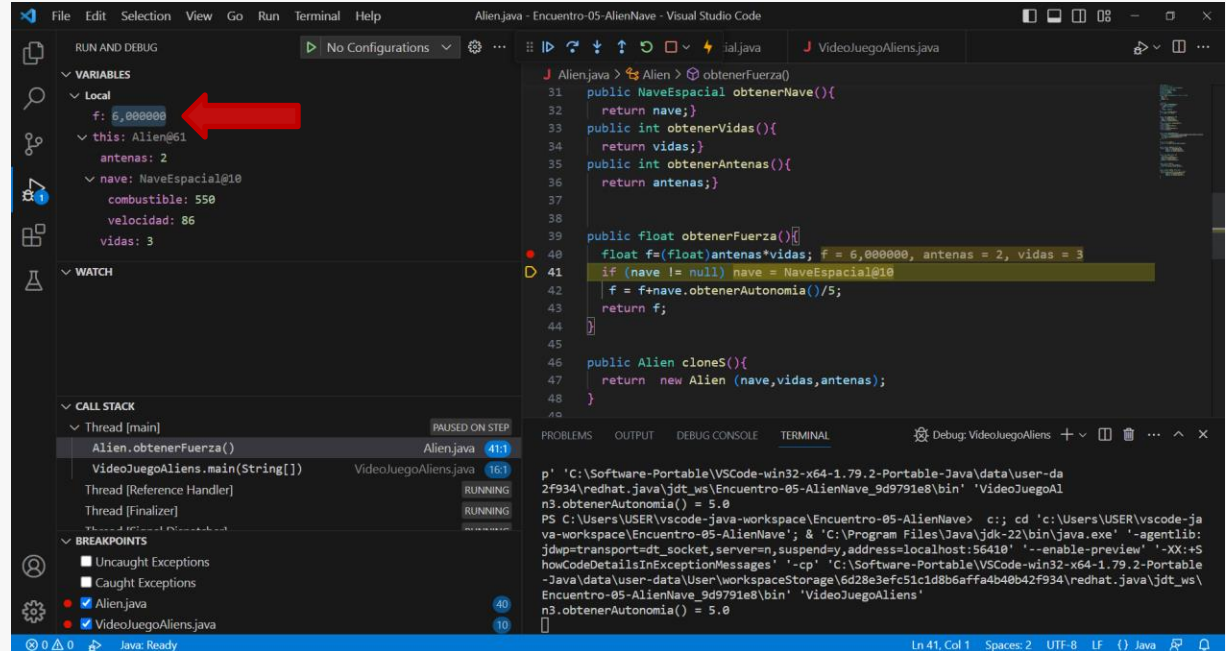
La recomendación es utilizar Step Into sólo en casos muy específicos, ya que es preferible colocar Breakpoints en el código a invocar que se desea analizar.

Modificando el contenido de memoria

5. El Debugger permite editar el contenido de entidades de tipos elementales y continuar la ejecución con esos nuevos valores.

En el ejemplo se declaró e inicializó la variable local `f` en 6.Of (como resultado de la asignación).

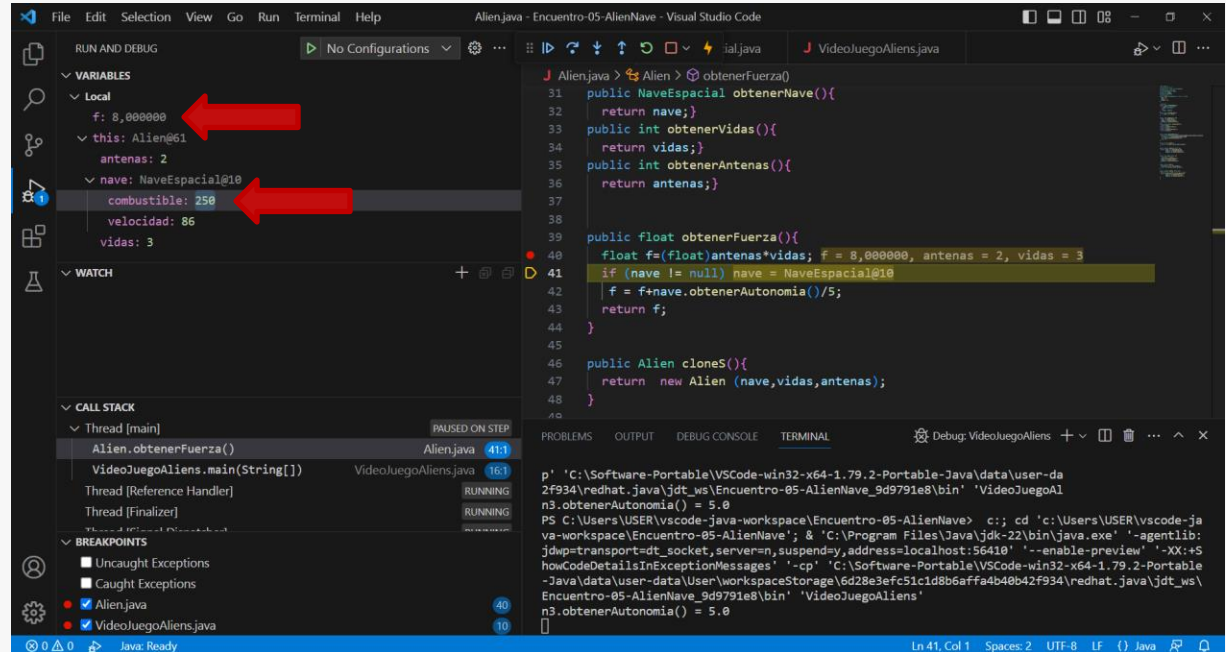
El debugger resalta las variables, atributos y parámetros recientemente actualizados para ayudarnos a ver qué fue lo último que cambió.



Modificando el contenido de memoria

5. El Debugger permite editar el contenido de entidades de tipos elementales y continuar la ejecución con esos nuevos valores.

Pero uno puede alterar esos valores y continuar la ejecución como si los mismos hubiesen sido generados como resultado de la ejecución.

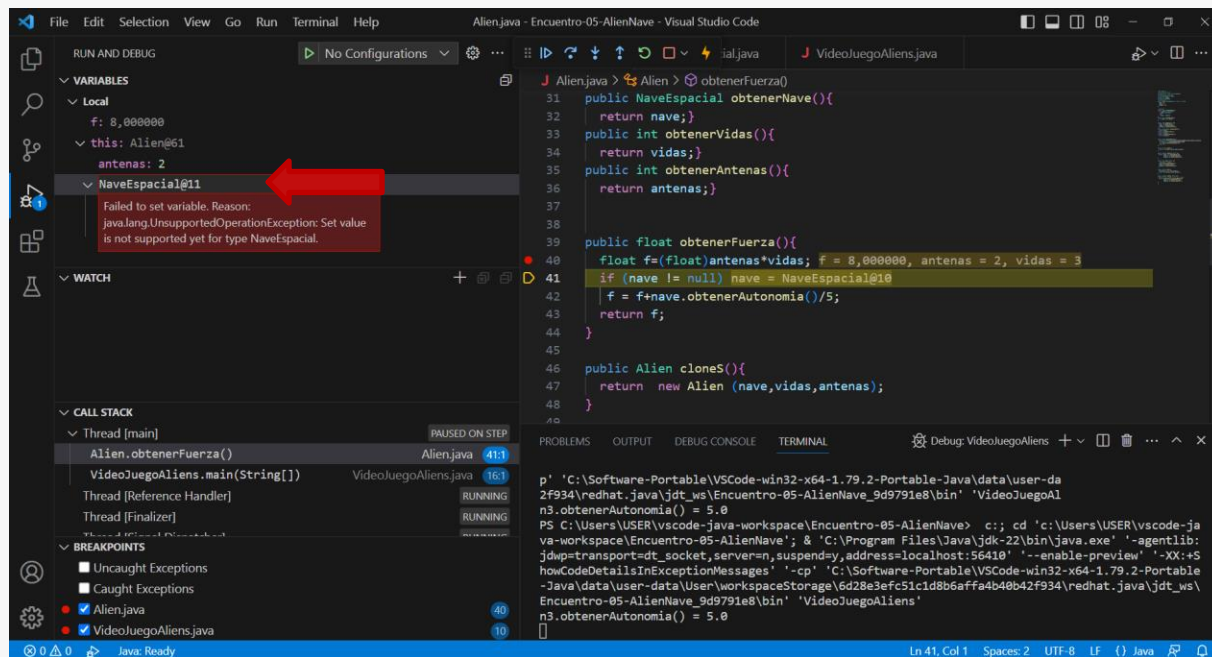


Modificando el contenido de memoria

5. El Debugger permite editar el contenido de entidades de tipos elementales y continuar la ejecución con esos nuevos valores.

No se pueden modificar referencias a objetos.

La notación **Clase@ID** en realidad es una representación textual de un objeto generada por la implementación del método `toString()` y no es posible alterar el objeto denotado intentando editarla.

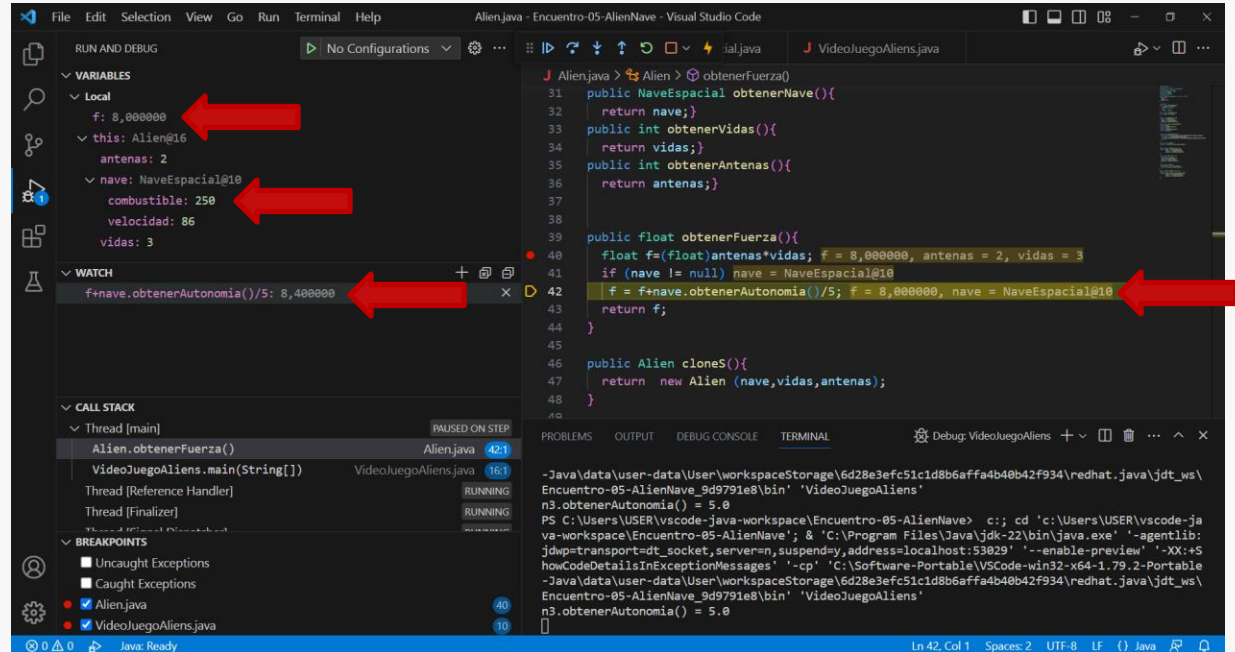


Modificando el contenido de memoria

5. El Debugger permite editar el contenido de entidades de tipos elementales y continuar la ejecución con esos nuevos valores.

La ejecución continúa con los valores alterados como si fueran los originales

Esto es útil para probar partes del código en casos que no se pudieron alcanzar a partir la ejecución normal (por errores previos o por ser casos no producidos por el contexto de ejecución).



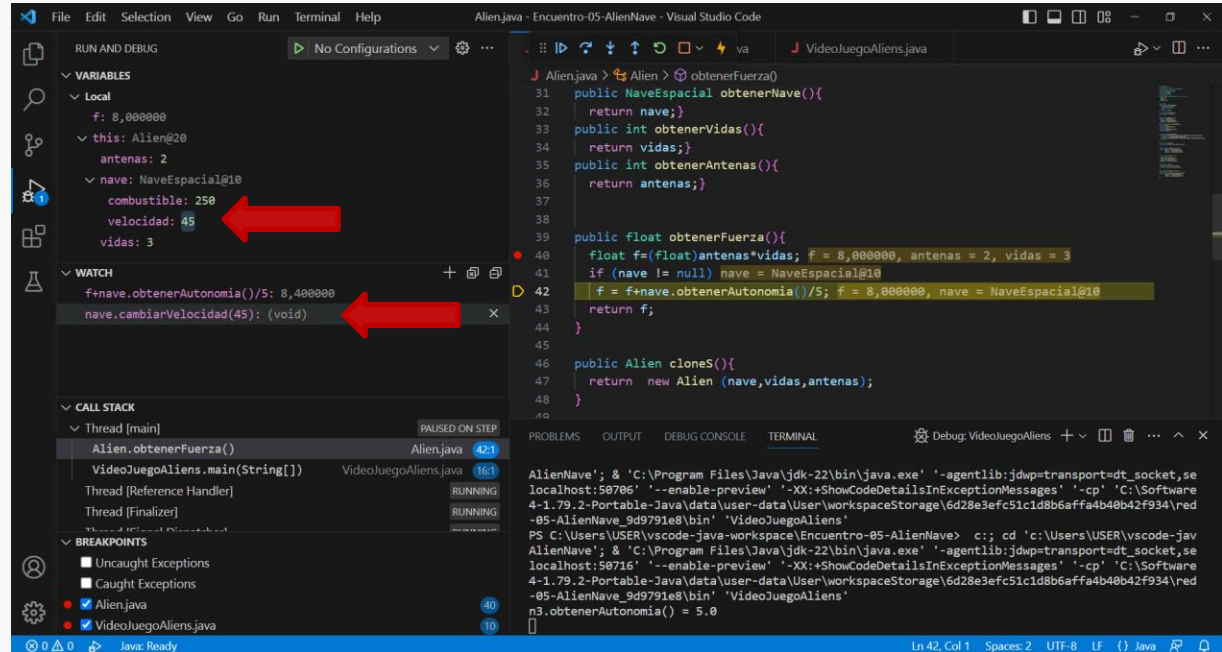
Modificando el contenido de memoria

5. El Debugger permite editar el contenido de entidades de tipos elementales y continuar la ejecución con esos nuevos valores.

También se pueden invocar comandos que produzcan efectos colaterales como expresiones de Watch y modificar el estado de los objetos por fuera del código de esta manera.

¡Mucho cuidado!

Cada vez que el debugger evalúe la expresión (ej. al editarla, al iniciar el debugger, **en cada paso de ejecución!**) se estará realizando la modificación.





Qué dejamos afuera en esta presentación

- Otras cosas que nos permiten hacer los debuggers son:
 - Establecer breakpoints condicionales
 - Establecer breakpoints basados en el disparo de excepciones y condiciones de error
 - Aplicar técnicas avanzadas de debugging basadas en testing unitario como la sustitución de objetos (dummies, stubs, mocks, etc.) que escapen al alcance de la materia y serán abordados en materias más avanzadas del plan de estudios.





Consideraciones y recomendaciones finales

- El debugger nos permite examinar en detalle la ejecución de nuestro código. Esto es muy útil para entender mejor cómo funciona Java, cómo implementa los conceptos de Orientación a Objetos que estudiamos en la materia, y también para encontrar errores.
- Una vez hallado el lugar del/los error/es, es recomendable:
 - **finalizar la sesión de debug,**
 - **introducir las correcciones necesarias en el código y**
 - **recién después volver a debuggear para confirmar** que los errores fueron corregidos (y no se introdujeron nuevos).
- Si bien los debuggers modernos soportan características como Hot Code Replace & Reload, es más seguro no realizar modificaciones de código mientras se debuggea para asegurar que el código ejecutado por el debugger y el que nos muestra el IDE sean consistentes.





Consideraciones y recomendaciones finales

- El debugger es una herramienta fundamental para el programador. Familiarícese con el mismo y aprendan a utilizarlo ya que les va a ahorrar muchísimo tiempo y esfuerzo de desarrollo y depuración.
- La principal ventaja del uso del Debugger es evitar ensuciar la lógica principal de nuestro código con instrucciones añadidas con el sólo propósito de conocer el estado de ejecución del mismo. El debugger nos permite ver el contenido de cualquier elemento de nuestro programa y seguir de manera detallada su flujo de ejecución. Úsenlo y dejen de lado técnicas de depuración más sucias tales como llenar de prints el código o comentar secciones enteras para tener idea de lo que está pasando.





¿Preguntas?

